

From-Date  $\Rightarrow$  when  $\times$  recharged  
TO-Date  $\Rightarrow$  current date

Agiled Jira - SDET QA

- $\rightarrow$  Git & Github  $\rightarrow$  SDET QA
- $\rightarrow$  postman  $\rightarrow$  step by Step Automation
- $\rightarrow$  Jenkins  $\rightarrow$  step by step Automation



\* watch Tests, Authentication pre-request Script, cURLs in "Automation step by step"

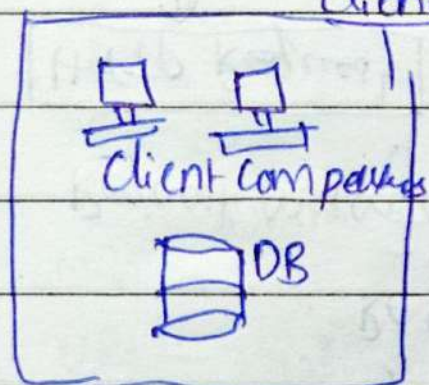
\* API stands for Application programming interface

client: client is a computer h/w or s/w that accesses a service made available by server

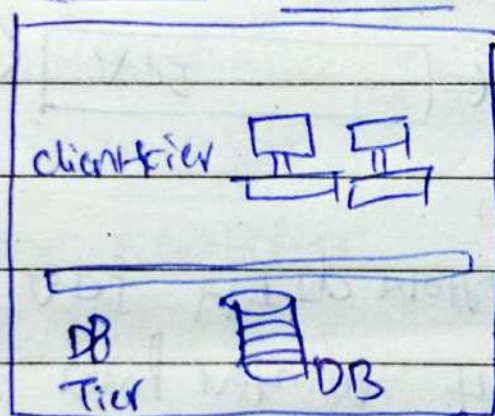
Server: It is a physical computer dedicated to run the services to serve the needs of other computers

\* It could be file server, database server, print server depending on the service that is running.

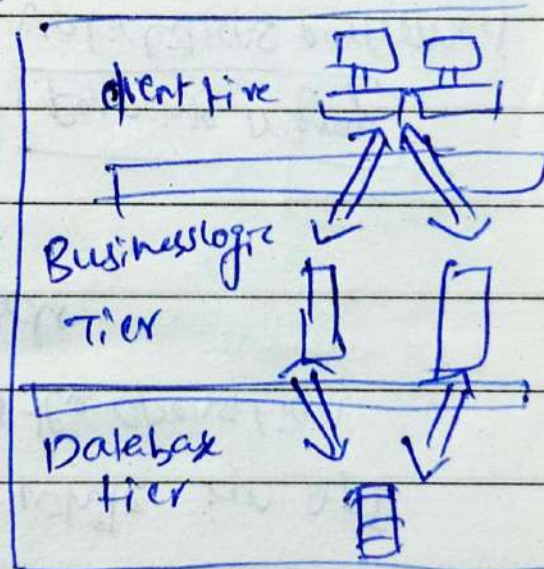
### Client Server Architecture



1-Tier Architecture



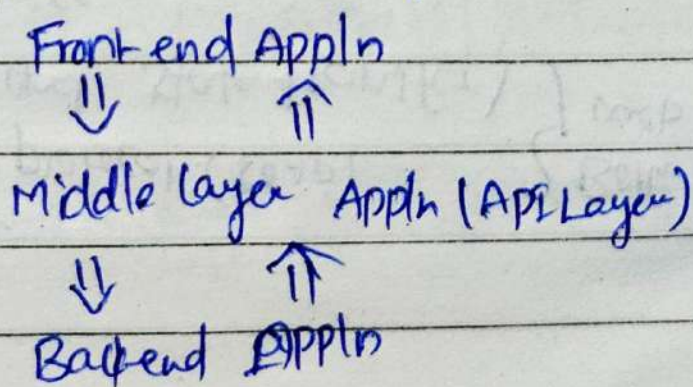
2-tier Architecture



3-Tier Architecture

\*\*\*  
API: API is the way of communication b/w two apps where applications may differ in their platforms or in terms of technology.

\* API is a way of communication b/w 2 layers.





## Types of API

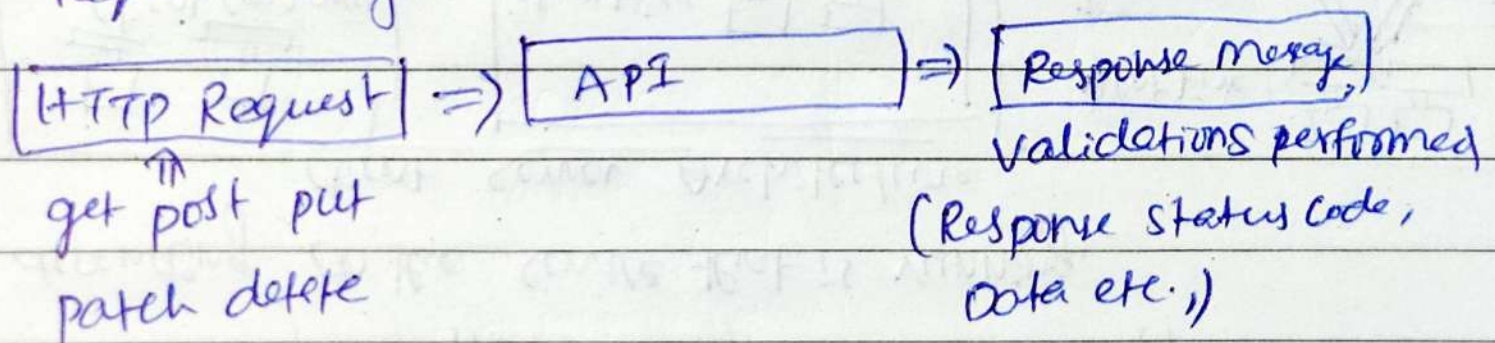
- 1) Simple Object Access protocol (SOAP)
  - 2) REST (Representational state transfer)
- } Both are web services

## API vs webservice

- \* webservice is an API wrapped in HTTP
- \* All webservices are API's but all APIs are not webservices.
- \* An webservice requires a network while an API doesn't require any network for the operation.

## REST API HTTP Methods

### Request message





# REST API Methods

| ↓ GET (I)                              | ↓ POST (II)            | ↓ PUT (E)              | ↓ Delete               |
|----------------------------------------|------------------------|------------------------|------------------------|
| Receive information about API resource | Create an API Resource | Update an API resource | Delete an API resource |

## HTTP vs HTTPS



http://www.ex.com  
password: abc123



without Encryption  
Hacker can see

## HTTPS



https://www.ex.com  
password: abc123



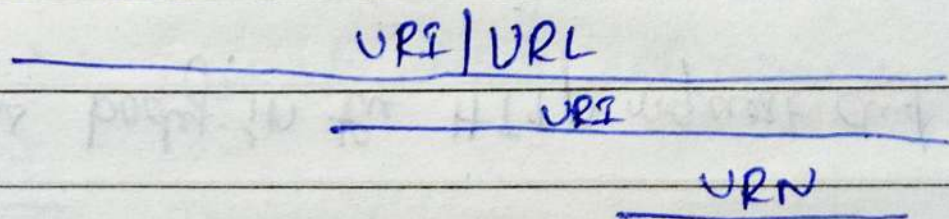
with password encryption  
Hacker see "Kyr abcdefg"

## Uniform Terminologies

URI - ~~User~~ Resource Identifier

URL - ~~User~~iform Resource locator

URN - Uniform Resource name



Scheme

Host

path

https://

Google.com/

Artifacts/Artifactname

Ex: CX → waiter order → Kitchen chef → waiter → CX



## Feature And Resource:

\* Feature is the term used in the manual testing to test some functionality

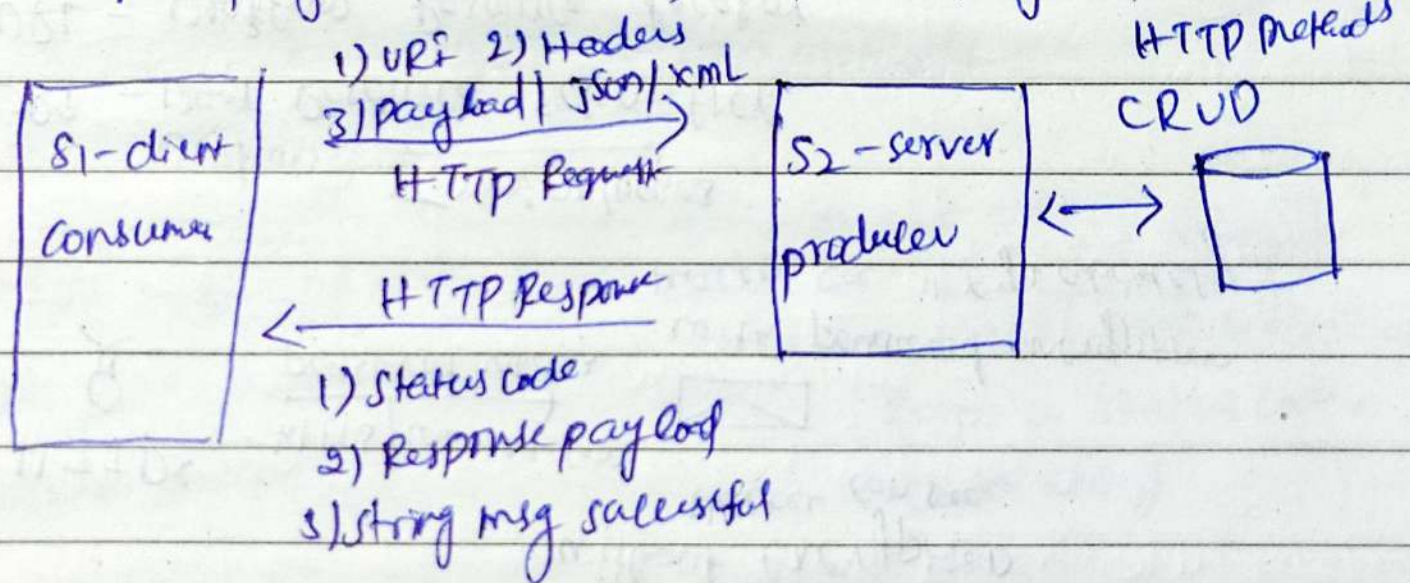
\* Resource is the term used in the API Automation testing referring to some functionality

payload

payload means body in the HTTP request and response message.

\* Request payload

\* Response payload



postman:

we can do manual testing of API's using postman desktop/web

- Download the postman from the Google chrome
- Once downloaded then create an account for the free



→ Create a workspace

workspace: Area where we maintain files and saved

Creating operations: Create workspace, rename, delete

Creating collection: Contains number of folders and  
http requests.

Operations: Create, rename, delete, run the collection

we can create any number under the workspace.

Request --> API --> Response

## HTTP Requests

- 1) GET - Retrieve the resource from the database
- 2) POST - create resource on database
- 3) PUT - update existing resource on database
- 4) DELETE - delete the existing resource from database
- 5) PATCH - update partial details of resource

## Sample API

https://reqres.in/api/users?page=2

Host

path  
parameters

query  
parameters



## HTTP Status Codes

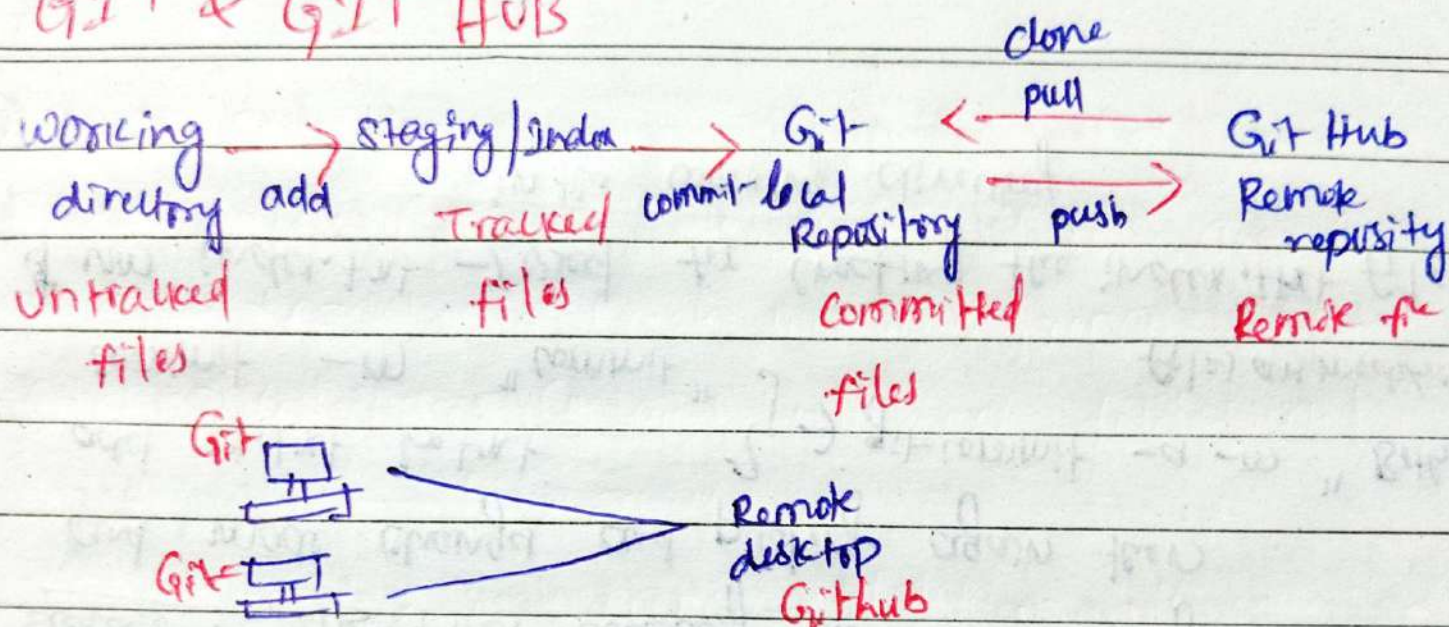
| Level 200                          | Level 400         | Level 500                  |
|------------------------------------|-------------------|----------------------------|
| 200: OK                            | 400: Bad request  | 500: Internal server error |
| 201: Created                       | 401: Unauthorized | 501: Not implemented       |
| 202: Accepted                      | 403: forbidden    | 502: Bad Gateway           |
| 203: Non-Authenticated Information | 404: Not found    | 503: service unavailable   |
| 204: No Content                    | 409: conflict     | 504: Gateway timeout       |
|                                    |                   | 599: Network Timeout       |

### Validations:

- 1) Status Code
- 2) time
- 3) size data
- 4) Response body (JSON/xml)
- 5) cookies
- 6) headers



# GIT & GIT HUB



=> Git is a distributed version control system designed to track changes in source code during the slow development

=> GitHub is used to push the tracked changes in the source code to the cloud during the slow development

Installation : Goto chrome -> Git Download -> Install

Git commands:

1) Init    2) Status    3) add    4) commit    5) log    6) Config

\$ pwd -> used for getting the current location in the cmd

\$ cd myproject -> changing the location to our workspace.  
current directory

\$ git init -> git has been initialized in the hidden file.

\$ git status -> used to get the commits yet & untracked files & present in project not yet added

\$ git add -a -> Used for adding all the files to git



\$ git add . \* → used for adding all the files

\$ git add a.txt b.txt → used for adding a & b txt files

\$ git status → used for getting the tracked files which are not yet committed and are to be committed.

\$ git config --global user.email "Bhanu@gmail.com"

↳ Identifying which one done configuration

\$ git config --global user-name "paran"

↳ Giving the name for identification

\$ git commit -m "first commit"

↳ Locally files are committed.

\$ git status → Defines that no commits to be done as commits done already.

\$ git log → used for getting how many commits are already done.

Scenario: AB files already added in locally to directory and made changes and pushing again then

add a.txt b.txt  
commit -m "commit" } → git commit -a -m "Both files are modified"

\$ vim index.txt → used for creating the index.txt file in the working directory.



\$ ls → used for showing files in the working area

\$ git ls-files → used for showing files in the git locally screening area

## Git DIFF Command:

git diff

working dir → staging → local repo → remote rep

\* diff command used for comparing the files b/w working dir to staging or staging to local repo etc.,

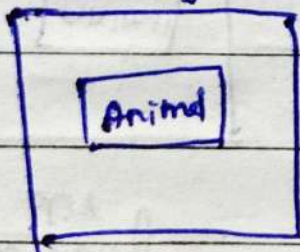
\* create a folder as myproject

open the myproject and click on right mouse open Git bash and create index.txt by using the following command.

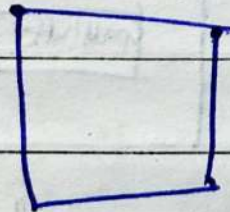
\$ vim index.txt

↳ write Animal in the file

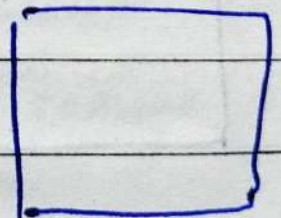
working dir



Staging



Local Repository



\* initializing the git and adding the file to the staging

\$ git ~~init~~ init → initializing

\$ git add • → adding all the files.



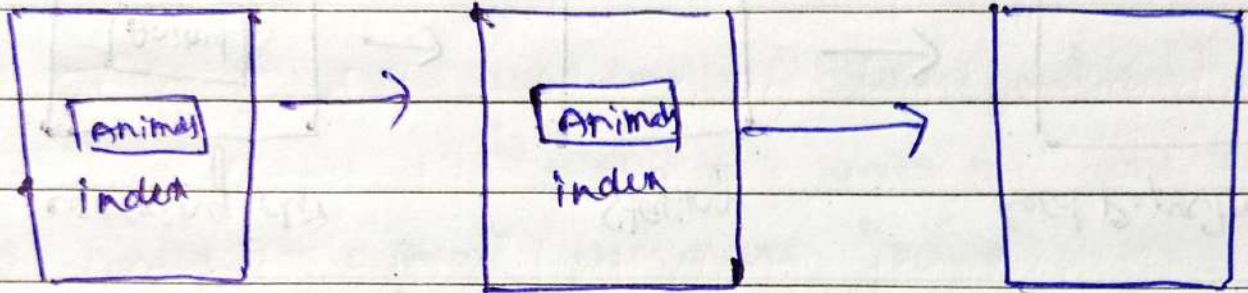
\$ git status → shows changes to be committed

~~Adding an~~

working dir

staging

local repo



→ Again opening the index file.txt and adding data (entra) to the file in the working dir.

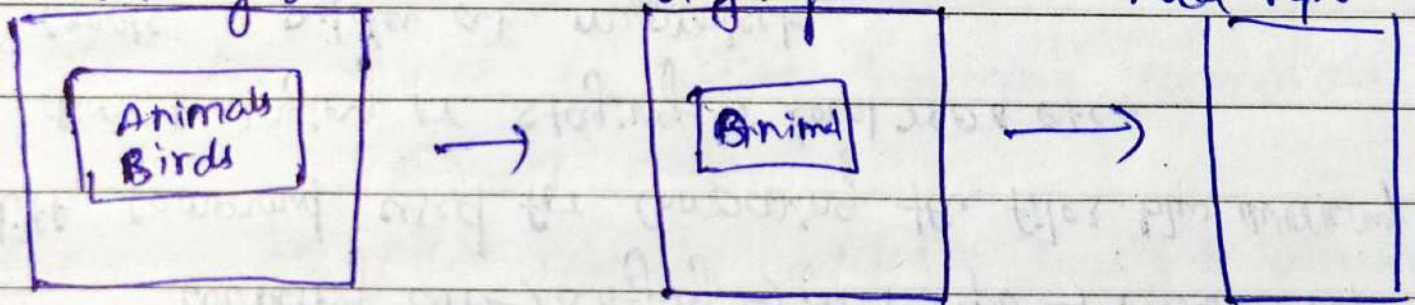
\$ vim index.txt

↳ Adding kat as Birds.

working dir

staging

local repo



\$ git status → defines the modified data in the index.txt

**Req 2:** To see the difference in file content b/w working directory and staging area

command: git diff index.txt



\$ git diff index.txt

o/p: diff --git a/index.txt b/index.txt  
index fcb5845..b5bf6bc 100644  
--- a/index.txt  
+++ b/index.txt  
@@ -1 +1,2 @@  
animal  
+ birds

### O/p Explanation:

a/index.txt → represents source (staging area)

b/index.txt → represents dest (working dir)

fcb5845 → Hash file content from src / staging

b5bf6bc → Hash of file content from dest / workspace

100644 → git file mode

100 → represents the file type

644 → File permissions

↓ ↓ ↓  
RW RP

4 - Read

2 - write

} permissions

1 - execute

--- a/index.txt → <sup>src file</sup> missing

Some lines (staging)

+++ b/index.txt → New lines added in the  
dest (working dir)

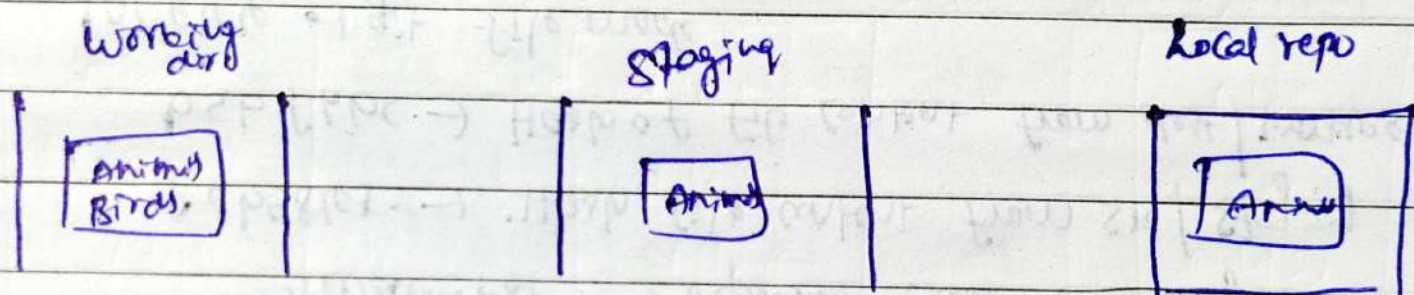
-1 +1, 2 } 1 line missing  
└──┬──┬──┘ 2 lines in dest  
└──┬──┘ Common  
└──┘ 1 line missing in src



If any line prefixed with space means it is unchanged  
if any line prefixed with + means it is added in dest copy  
if any line prefixed with - means it is removed from  
dest copy.

**Req 2:** To see the diff in file content b/w working directory  
and Last Commit

\$ git commit -m "first commit file contains 1 line"



first commit file  
contains 1 line.

\$ vim index.txt

↳ adding fishes

\* Last Commit referred using HEAD

command: `git diff HEAD index.txt`

**o/p:**

Animals → unchanged  
space  
+ birds → } added in the destination  
+ fishes → }



**Req 3:** To see the difference in file content b/w

staged copy and Last Commit

| working dir                | staging | Local repo |
|----------------------------|---------|------------|
| Animals<br>Birds<br>fishes | Animals | Animals    |

\$ git add . → adding working data file to staging

| working                    | staging                    | Local repo |
|----------------------------|----------------------------|------------|
| Animals<br>Birds<br>fishes | Animals<br>Birds<br>fishes | Animals    |

command: `git diff --staged HEAD index.txt`

\$ git diff --staged HEAD index.txt

op:

- animals → unchanged  
+ birds } → staging area has  
+ fishes } added files.

**Git RM command**

→ open Gitbash and create file1.txt, file2.txt and also file3.txt by using the following commands.

\$ vim file1.txt  
↳ add text

\$ vim file2.txt  
↳ add text

\$ vim file3.txt  
↳ add text



~~\$ git init~~ ls → retrieves all files in the working directory

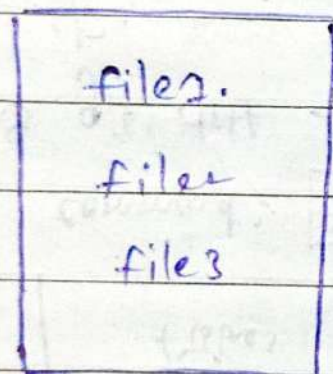
\$ git init → git initializing

\$ git add . → Adding all the files to the git

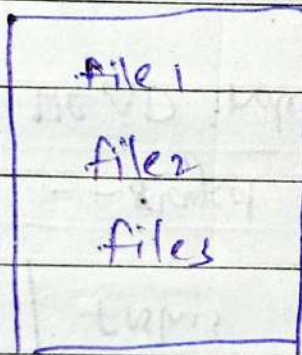
\$ git commit -m "first commit" → pushed into the Local repository

\$ ls → files.txt file2.txt files.txt & working dir files

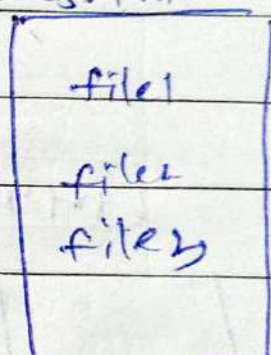
\$ git ls-files → retrieves files in the local repository  
↳ files.txt file2.txt files.txt



working directory



staging Area



local Repository

**Req 1:** Remove files from both staging & working directory

**git rm files.txt**

\$ git rm files.txt → removes files from working dir and ~~rep~~ staging dir

git rm -r . → Used for removing all the files from staging and working directory



Req 2: Remove files from only staging

`git --cached file2.txt`

\$ `git --cached file2.txt` → used for removing files from staging area.

Req 3: Remove files only working directory

`rm file3.txt`

\$ `rm file3.txt` → used for removing file from working dir

## GIT CHECKOUT Command

We can use checkout command to discard unstaged changes in the tracked files of working directory

1) Only for working directory

2) To discard unstaged changes (The changes which are not added to staging area)

3) In the tracked files (The files which already added to staging area/commit)

⇒ open gitbash and create a new file.

\$ `vim file1.txt`

↳ add data as This is first line

This is second line.

\$ `cat file1.txt` → used for working dir file data fetching

↳ Displays This is first line

This is second line



\$ vim files.txt

↳ adding 2 line again

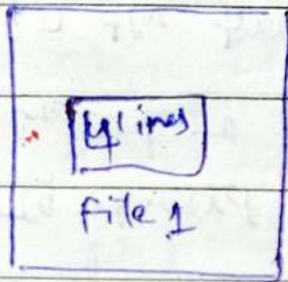
[ This is first line  
- This is second line  
- This is third line  
This is fourth line

\$ cat files.txt → displays the above four lines

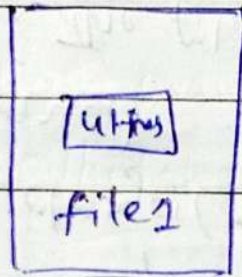
\$ git init

\$ git add .

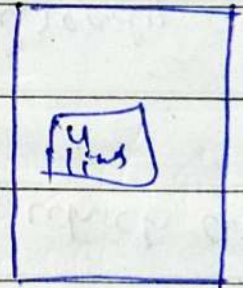
\$ git commit -m "first commit"



working dir



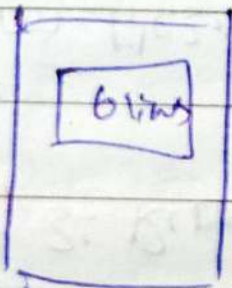
staging



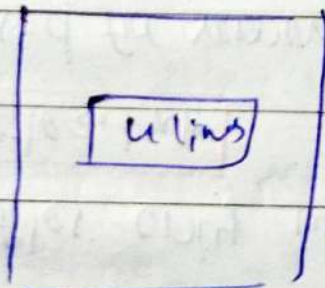
Local repo

\$ vim files.txt

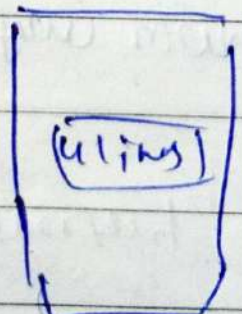
↳ Add other lines → This is fifth line  
This is sixth line.



working dir



staging



Local repo

\$ cat files.txt

↳ Displays 6 lines.

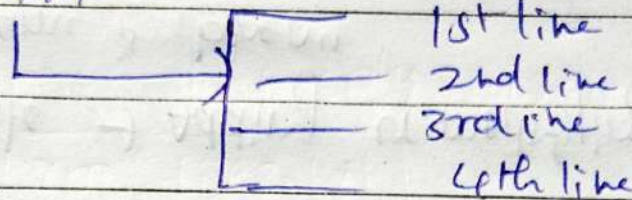


## git checkout -- file1.txt

→ \$ git checkout -- file1.txt

↳ Removes the 2 lines in working dir

\$ cat file1.txt



\*\*\* This process applicable only when file1 added to staging area or local repository. \*\*\*

Ex: ~~git~~ ~~rm~~ file2.txt → created new file in the working dir

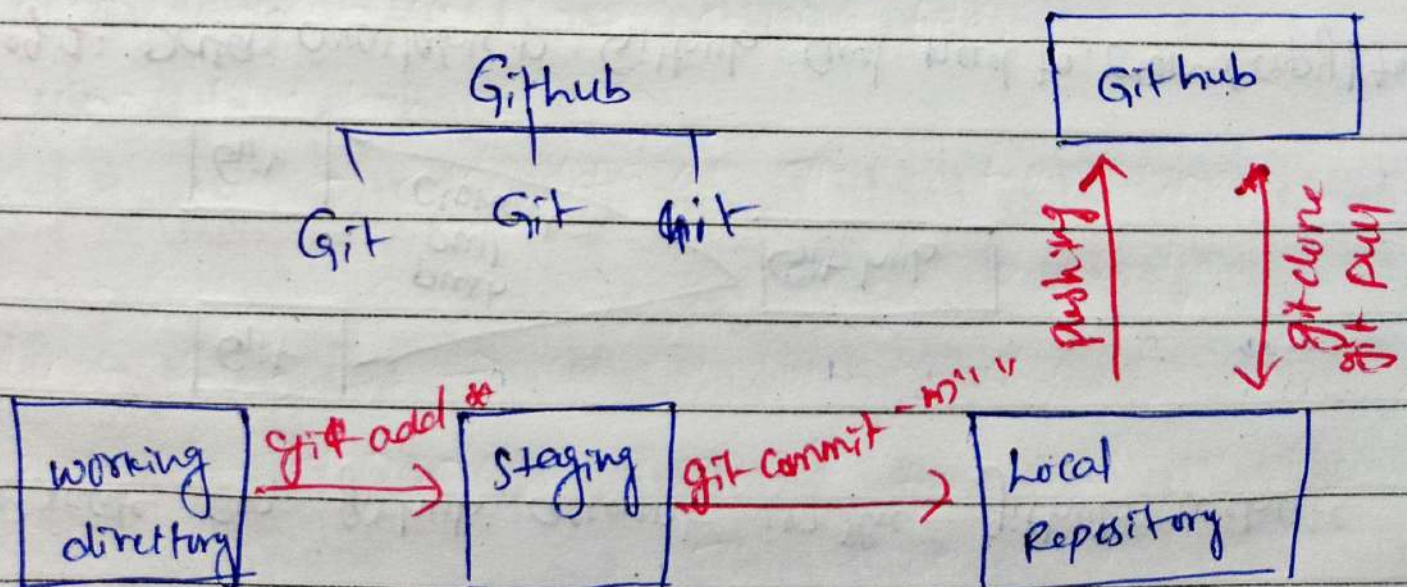
\$ git checkout -- file1.txt

↳ Error since its not added to staging or local repo area.

## GIT HUB:

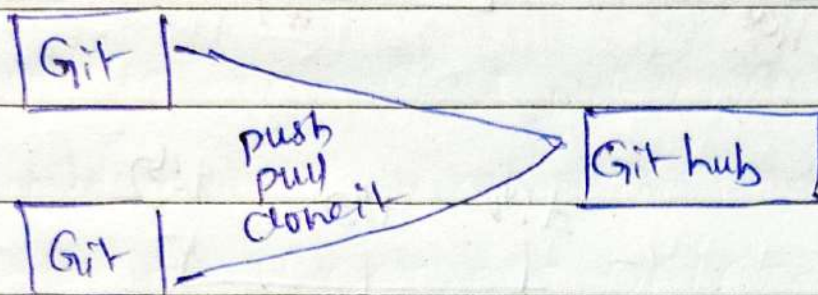
→ Github is a hosting service for git repositories

→ Git is a tool while Github is the service to use git





→ create an github account in the github website



**Req 1:** Data available in Github and need to clone (copy) to local system and continue to work on the same

→ Create any folder (Desktop / new folder)

↳ Open Gitbash over there clone it

```
$ git clone 'https://github.com/pawantraing/test-repo.git'
```

Repository URL

**Req 2:** Modifying cloned files and also for adding other file.  
\$ ls → for getting cloned file. and pushing to github back  
my file

\$ cat my file → retrieving data from file in working directory  
my name is bhannu

\$ vim my file → Adding or modifying data  
my name is bhannu  
I am from MPL

\$ vim my file 2  
I am working as developer



\$ git init → git initialization

\$ git add . → adding files to staging

\$ git commit -m "first commit" → data pushed to the  
Local repository

\$ git push origin main → files pushed to github (remote).

### Git pull

Getting missing  
data of files in working  
dir from the Github.

### Git clone

Copying all files from  
Github

**Req 3:** Data modified in the Github (remote repo) and  
working dir has already cloned from git and need to  
get the updated data.

\* Data changed in the file of repo in Github

\* Data will not change in the working repo which  
is already cloned. for getting modified data we  
use following command.

\$ git pull origin main → pull data from github  
to working repo

\$ cat my file.

↳ we will get updated data from github to  
working dir and show the same



## REST Assured:

- REST Assured is a Java library for testing RESTful web services
- It is used to invoke REST web services and check response
- Can be used to test xml as well as JSON based web services
- It supports POST, GET, PUT, DELETE, OPTIONS, PATCH and HEAD requests and can be used to validate and verify the response of these requests.
- Can be integrated with testing framework like JUnit, TestNG etc.,
- REST Assured is implemented in Groovy
- Uses BDD style for testing:
  - \* Given                      \* when                      \* Then

→ REST Assured website

<http://rest-assured.io>

### prerequisites:

1) Java                      2) IDE (Eclipse, IntelliJ, etc.,)                      3) Maven

### 4) TestNG

1) creating first project

→ click on open RestAssured

→ create a maven project

→ Add RestAssured dependency, TestNG dependency

→ save the project



→ Check all dependency jars downloaded or not.

→ Install TestNG plugin in the eclipse market place

401 Unauthorized error:

↳ We need to add the Header key

1) Create 1st project → 5) Add Assertions

1) Create a class 6) Run & verify

2) Create a function and annotate with @Test (TestNG)

3) Run a GET request

4) Store response and print response data

→ src/test/java → coding done.

```
public class Test
```

```
{  
    @Test
```

```
    public void m() {
```

```
        Response response = RestAssured.get("url");  
        // class name → class
```

```
        System.out.println(response.getStatusCode());
```

```
        System.out.println(response.getTime());
```

```
        System.out.println(response.getBody().asString());
```

```
        System.out.println(response.getStatusLine());
```

```
        System.out.println(response.getHeader("content-type"));
```

```
        // Assert.ans int actual-code = response.getStatusCode();
```

```
        Assert.assertEquals(actual-code, 200);
```

```
    }
```

```
}
```